

In this feature, industry experts discuss the benefits of using low-power microcontrollers, as well as the advantages of modern-reduced power modes and power-budgeting techniques. They also explore the relative merits of selecting a microcontroller based on its target application or on development support. Abhishek Mutha of *EFY* got in touch with industry experts from Microchip, Freescale, Texas Instruments and Renesas, who shared their thoughts on today's microcontrollers—how they have evolved and what benefits they offer design engineers who use them in their projects. Combining all their responses, we bring to you a broad perspective

In many applications, the microcontroller does not run continuously and may be 'sleeping' much of the time. How have sleep modes evolved during the years?

Rather than having two modes, i.e., RUN and SLEEP, modern microcontrollers have a spectrum of modes in between, which offer varying degrees of functionality.

On the PIC microcontrollers with eXtreme Low Power, the modes are named Doze, Idle, LVSleep, Sleep and Deep Sleep. Doze allows the peripherals to be clocked at a higher rate to meet communication needs or timer sequences. But it allows the core to run at much lower speeds, reducing the current consumption by up to 75 per cent. Idle turns the core off completely. LVSleep, Sleep and Deep Sleep turn off all peripherals but preserve select features like the LCD, analogue-to-digital converter (ADC), timers and interrupts. So a lot can be accomplished even in these modes to save 97-99.9 per cent of the current compared to the RUN mode.

Real-time low-energy environments pose special challenges for the designer and programmer, because the same sleep states that reduce a microcontroller's energy consumption often also reduce its ability to quickly respond to an event. Low-power modes typically range from a light sleep or standby mode, through deep sleep. Generally, the deeper the sleep, the lesser the power consumed by the microcontroller. A number of strategies are used in projects to determine how these variables can be set to take the best advantage of a microcontroller's sleep modes. One is to identify the routine low-level functions required by the application and identify microcontroller architectures that can handle as many of these functions as possible, using on-chip hardware that does not require CPU intervention. Other methods are clock gating, gate level optimisation, voltage scaling and digital architecture techniques.

Basically, if the system is designed intelligently and if the microcontroller architecture supports this, power consumption can be reduced significantly. For example, most microcontrollers now have direct memory access (DMA), and if a data transfer function has to be done, the core can actually be kept in OFF mode, which saves substantial power. Also, the time the internal phase-locked loop (PLL) or frequency-locked loop (FLL) takes to stabilise from low power to active mode determines the average power consumption.

There are standard techniques of reducing power by optimising the voltage levels and clock frequency. Attention also needs to be paid to the IO lines leakage (including communication interfaces), which can help in substantial power savings.

Sleep and standby modes have certainly evolved over the years. In Renesas' RL78 series, we have showcased this evolution. In this product, we implemented three types of sleep modes. These are (i) HALT mode, (ii) STOP mode, (iii) SNOOZE mode. SNOOZE mode is especially interesting, because it allows the operation of some peripherals like analogue-to-digital converter (ADC) and universal asynchronous receiver/transmitter (UART), while in standby mode. Besides taking advantage of the various microcontroller sleep modes available, designers can moderate the CPU performance by switching off or scaling back the clock speed whenever possible.

They can further reduce microcontroller power consumption by turning off any clocks and peripheral blocks that are not being used.

What are the most exciting features that have come up to delight engineers—in the software for programming the microcontroller or the embedded peripherals within the microcontroller?

Low-power microcontrollers are now available with 16-bit sigma-delta ADCs built in. For communications, we are seeing USB, controller area network (CAN) and IC-to-IC communication being adopted in many ways. And, finally, we are seeing small bits of programmable logic being added. This is great, since you can now stitch together peripherals in the manner you want within the microcontroller, and have it operate in sleep mode at tiny current levels.

In terms of peripherals, microcontrollers now come with high-resolution ADCs (up to 24-bit sigma-delta ADC), high-resolution DACs, USBs, Ethernet, high-speed pulse width modulators, real-time clocks (RTCs), LCD controllers, etc. In terms of programming, these microcontrollers offer integrated drive electronics (IDE) and good compilers. They also offer real-time operating systems free with 32-bit microcontrollers.

From a functionality perspective, we see analogue peripherals optimised for certain end equipment or applications being increasingly adapted by most suppliers, making them a cost-effective implementation (op-amps, 16-24 bit ADCs, DACs). Also, connectivity is gaining ground across embedded applications, and technologies like Ethernet, controller area network (CAN), USB and RF are increasingly being offered.

Today, functions that are 'smart' and 'connected' are much sought-after functionalities in microcontrollers. For example, in the Renesas RL78 Smart Analog series, the analogue front-end is integrated into the microcontroller. It is a new technology that allows interfacing to hundreds of sensor types with a single, configurable analogue front end (AFE) platform—resulting in more intelligent sensors with a smaller footprint, while significantly reducing development time.